

Designing Discriminative Physics Benchmarks for AI Coding Agents: Evidence from Two Data-Driven Analysis Tasks

Jiaqi Zhu^{1a}, Jim Huang^{1b}, Anson Lin^{1c}, Jason Yang^{1d}

¹NeoSchool Research Program

Mentor: Gunther Roland, Massachusetts Institute of Technology; Teaching Assistant: Alex Cai

^ajz6553@nyu.edu, ^bjby235278@gmail.com, ^cjxyxf666@bu.edu,

^d2792463507@qq.com

Abstract— Agentic coding systems are increasingly used for data-driven scientific analysis, yet a benchmark can lose its measurement value when capable systems converge on the same headline answer. We study how to preserve discrimination in supervised physics evaluation, where hidden ground-truth targets define correctness even when the agent does not receive all target labels during training. We compare a damped-harmonic-oscillator control task with a particle-identification task under a common core dispatch and scoring framework, reporting 51 analyzed runs: 15 runs from five command-line agent interfaces in Experiment 1 and 36 runs from four interfaces in Experiment 2; Kimi CLI did not complete Experiment 2 and is excluded from its quantitative analyses. In the second task, the event-level dE/dx estimation target is withheld from the agent-visible training data; this preserves a supervised evaluation target while testing whether the agent can infer the measurement process rather than learn a direct regression shortcut. Deterministic metrics carry most of the score, a separate judge track assesses the submitted workflow, and full marks are calibrated to an empirically measured reference floor. The control task saturates at numerical precision and therefore ranks agents mainly through cost. The redesigned task leaves classification scores tightly clustered but spreads dE/dx estimation scores from 19.8 to 86.6, more than a factor of four. The comparison supports three design principles: introduce a scientifically meaningful failure path that the headline metric cannot reveal, calibrate full marks to an achievable measured floor, and report dimension-level contributions rather than only a weighted total. The evidence is limited to the agents, task conditions, and model versions studied here.

Keywords- AI coding agents; scientific benchmark design; supervised prediction; benchmark calibration

1. Introduction

Agentic coding systems can inspect files, write and execute analysis code, revise a model after observing its output, and return both a numerical result and an auditable workflow. These capabilities make them plausible assistants for data-driven science, where the central difficulty is often not syntax but deciding what to measure, which assumptions are defensible, and how to validate a result when the data are imperfect. Physics is a useful testbed because its models impose quantitative constraints and its

simulated or held-out targets permit objective checks of numerical output and scientific plausibility. In this paper, both experiments are supervised prediction tasks: the evaluation server retains a fixed ground truth and computes scores without relying on an evaluator’s impression. The benchmark can therefore test an auditable workflow while preserving a clear definition of correctness.

Existing evaluations provide important pieces of this picture. SWE-bench evaluates whether an agent can repair a real software repository against executable tests; ScienceAgentBench packages data-driven scientific workflows as self-contained programming tasks; and PRBench and Collider-Bench extend evaluation toward end-to-end reproduction of computational physics analyses. LiveBench shows why benchmark maintenance and contamination control matter when models improve quickly. These benchmarks ask whether an agent can complete a specified task under a specified protocol. Our question is different: how should a supervised scientific task be designed when the principal target has become easy? Once all systems reach the same target, adding more scoring dimensions cannot recover information that the task never elicits.

We therefore ask: how should a supervised physics benchmark be constructed so that it continues to distinguish scientific analysis after its principal prediction has become easy? We answer with two sequential experiments implemented within a common core framework. Experiment 1 is a deliberately transparent damped-oscillator forecasting task. Experiment 2 retains the core agent-dispatch and two-track scoring architecture but introduces a redesigned task and documented operational changes, including a longer timeout, fresh working-directory recreation, and revised prompt handling. Four interfaces completed Experiment 2; Kimi CLI did not complete the headless protocol and is excluded from its quantitative analyses. Three interfaces ran their unpinned default models, and two failed runs were re-executed; these deviations are detailed in Sections 5.2.3 and 5.2.4. The particle label is supervised through hidden ground truth, whereas the event-level dE/dx target is withheld from the agent-visible training data. This separation allows the benchmark to score a correct estimate while testing whether the agent can reconstruct what the measurement represents.

Our contribution is methodological rather than a new leaderboard. First, we show that a benchmark can satisfy conventional validity criteria yet collapse into a price comparison when its substantive dimensions saturate. Second, we show how a hidden measurement-level output, combined with empirically calibrated scoring, can restore discrimination without replacing the command-line interfaces under test or the core agent-dispatch implementation. Operational deviations and model-pinning limitations are reported explicitly rather than treated as identical execution conditions. Third, we demonstrate why conclusions should expose the spread and uncertainty of each scoring dimension, the cost of obtaining it, and the failure modes behind it. The resulting claims concern benchmark construction and recalibration; they are not a general capability ranking or a procurement recommendation for the five interfaces tested.

2. Related Work

Benchmarks for agentic science increasingly require models to act through code and produce an executable scientific artefact rather than answer an isolated question. ScienceAgentBench derives 102 tasks from 44 peer-reviewed studies and evaluates program quality, execution outcomes, and cost [1]. PRBench evaluates end-to-end reproduction of quantitative results from published physics papers [2], while Collider-Bench focuses on particle-physics analysis reproduction and checks the fidelity of generated analysis outputs [3]. These studies establish the value of hidden ground truth, controlled execution, and multidimensional scoring. Our work is complementary: it does not ask agents to reproduce a paper, and it is not a collection of unrelated domain tasks. It holds the agent-visible protocol fixed while measuring how task redesign changes what a supervised benchmark can distinguish.

A separate family of benchmarks probes physics reasoning rather than complete data-analysis workflows. TPBench uses novel problems in high-energy theory and cosmology spanning undergraduate to research level [4]. PhySense targets principle-first physical reasoning, emphasizing whether a model identifies the governing idea rather than merely producing a long solution [5]. PhysUniBench broadens coverage to multimodal undergraduate problems with diagrams and graded difficulty [6], while NewtonBench studies data-driven discovery of generalizable scientific laws [7]. These efforts test important forms of conceptual, mathematical, and multimodal competence. The present tasks instead begin with files of measurements and require an autonomous, reproducible prediction pipeline with withheld numerical targets. They remain supervised evaluation tasks because the target is fixed and scored after submission, even though the dE/dx estimation target is absent from the agent-visible training data. The central question is whether task construction exposes a failure that final-answer accuracy conceals.

General agent evaluation also supplies design principles relevant here. SWE-bench made real repositories, isolated environments, and executable tests a durable model for evaluating coding agents [8]. LiveBench addresses contamination and benchmark obsolescence through frequently refreshed, automatically scored tasks [9]. HELM argues for reporting multiple axes, including efficiency, rather than treating accuracy as a sufficient summary [10]. We adopt the same measurement discipline: the task files and environment are controlled, validity is treated as a gate, cost is recorded separately, and repeated runs are used to quantify stochastic variation. The distinction is that our feedback loop operates at the level of scientific task construction: the result of the control experiment determines how the second task is designed.

Finally, we use model-based judging only as a secondary source of evidence. MT-Bench and related analyses show that LLM judges can be useful for open-ended comparisons but are sensitive to position, verbosity, and other presentation effects [11]. Accordingly, deterministic metrics against hidden targets carry the dominant weight in both experiments, while the judge reviews the submitted code, narrative, and self-report under an explicit rubric. This separation follows the broader lesson from agent evaluation: a single aggregate score is not enough to explain a ranking. The contribution here is a construction and recalibration procedure for keeping a supervised scientific benchmark informative, not another static task suite.

3. Data and Materials

3.1 Damped harmonic oscillator

The control task uses a public dataset of a damped harmonic oscillator obtained from Kaggle [12], distributed already partitioned into a training and a test set. Each file contains four columns (time, displacement, velocity and acceleration) with 200 and 300 rows respectively. The dataset is used as supplied; no resampling or re-partitioning was performed.

3.2 Particle identification from dE/dx

The second task uses a simulated detector sample generated by our mentor’s group, supplied as `dedx_sample_ecal_flatpT_pikpde_28_100000.txt` (27,248,608 bytes). It contains 100,000 events, each a single charged particle of one of five species (electron, pion, kaon, proton and deuteron), present in near-equal numbers of approximately 20,000 each. Each event header carries a single momentum field, uniform over 1–10 GeV/c.

Two properties of the generator were confirmed with the provider after the runs had been scored, and both bear on how the sample should be read. The momentum field stores the transverse momentum, but every particle was generated at pseudorapidity zero, so $p = p_T$ exactly and the argument of the Bethe–Bloch curve, $\beta\gamma = p/m$, may be evaluated directly from the stored field; we write it as p throughout.

Second, the withheld energy-loss column is not a statistic of an event’s fifty measurements at all. It is the mean energy loss given by the provider’s parametrisation of the Bethe–Bloch function at that particle’s velocity, and the fifty measurements are noisy draws about that mean. The target therefore carries no sampling noise of its own, which is what makes the estimation sub-task well posed: an agent is asked to recover a noiseless quantity from fifty noisy samples of it, and the smallest achievable residual is a property of the sampling rather than of the target.

The file is ragged and semicolon-separated: one five-field header row per event, followed by that event’s individual energy-loss measurements, one per row, of which every event carries fifty. The complete schema appears in Table 1 and the first event is reproduced in Appendix A.

The generator itself, together with its version, random seed and configuration file, was not supplied to us; the filename and the provider’s description above are the provenance metadata available. One further property was recovered by measurement rather than stated: the rounding precision of the stored mass values (§4.3.1). Appendix B gives a reconstruction procedure and separates what the data verify from what remains undisclosed.

Table 1. Schema of the dE/dx sample.

Row type	Field	Type	Unit	Range observed	Given to agent
Header	n_hits	int	—	50 (all events)	implicitly
Header	momentum	float	GeV/c	1.001 – 9.997	yes
Header	dE/dx prediction	float	keV/cm	—	no
Header	mass	float	GeV/c ²	0.0005, 0.139, 0.494, 0.937, 1.874	no
Header	label	int	—	0, 1	no
Sample × 50	dE/dx	float	keV/cm	0.473 – 70,700	yes

Header fields are semicolon-separated on a single row. Masses are stored rounded to three or four decimals, which is why species assignment uses an absolute-or-relative tolerance (§4.3.1). The two withheld header fields are the species label itself and the ground truth for the estimation sub-task. The momentum field stores the transverse momentum, which equals the total momentum p because the generator set pseudorapidity to zero (§3.2).

4. Methods

4.1 Framework

The benchmark is constructed in four stages (Fig. 1). Stage 1, task selection, fixes what the benchmark can measure. Stage 2, data and prompt preparation, fixes what the agent may see: the dataset is split, targets are masked, and an instruction is written that describes the data without prescribing a method. Stage 3, headless execution, fixes what the comparison holds constant. Stage 4, scoring and ranking, fixes what the ranking is made of. The dashed return path closes the loop: the outcome of Stage 4 measures the task as well as the agents, and fed back into the design of the second experiment.

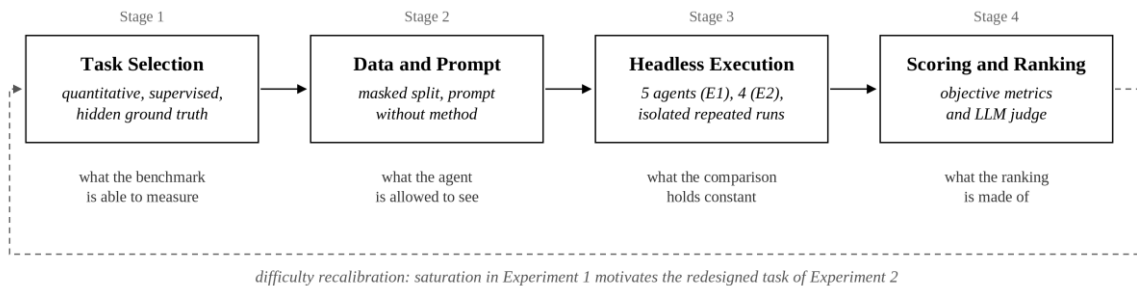


Fig. 1 Four-stage benchmark construction pipeline.

4.1.1 Task eligibility

Every task admitted is data-driven rather than text-based, so that an answer is a number; supervised, so that a correct answer exists independently of the evaluator’s judgement; and has its ground truth withheld until scoring, so that success cannot be obtained by retrieval. These conditions exclude open-ended derivation, which cannot be scored deterministically, and reproduction of published analyses, where the target is discoverable.

4.1.2 Unit of comparison

The units of comparison are agentic command-line interfaces rather than the models beneath them. Each wraps a model in its own scaffolding (file access, shell execution, multi-turn planning, background task management), so a score reflects the combination. This is motivated by the failure modes that matter for delegated work: an agent that identifies the correct physics but dispatches its analysis to a background process and then ends its turn, or one that cannot be launched without a terminal, has failed the task irrespective of its model.

Five interfaces were selected to span providers rather than to be exhaustive (Table 2). All are generally available, all support a headless mode, and together they cover roughly an order of magnitude in per-run cost.

Table 2. Agentic command-line interfaces under test.

Agent	Provider	Model invoked	Pinned	Runs (E1 / E2)
Qwen Code	Alibaba Cloud	qwen3-coder-plus	Yes	3 / 9
Kimi CLI	Moonshot AI	kimi-k3	Yes	3 / 0
Claude Code	Anthropic	claude-opus-4-8	No	3 / 9
Codex CLI	OpenAI	gpt-5.3-codex	No	3 / 9
CodeBuddy CLI	Tencent	GLM-5.2	No	3 / 9

“Pinned” indicates the model was fixed by this study rather than inherited from the interface default; unpinned entries record the model actually invoked, read from the run logs. Interface versions: Qwen Code 0.20.0, Kimi CLI 0.29.2, Claude Code 2.1.74, Codex CLI 0.145.0, CodeBuddy CLI 2.125.0 [15].

4.1.3 Information control

Quantities used for scoring are removed from every file the agent can read. Where a derived quantity would permit the withheld target to be reconstructed, it is withheld as well; the predicted energy loss in Experiment 2 is withheld on exactly this ground. The instruction describes the physical meaning and units of every supplied column and nothing about method. Domain identity cannot always be concealed, since the training labels read “pion” and “kaon”, but the solution path can be, and that is what is under test.

4.1.4 Scoring architecture

Scoring is divided into a deterministic objective track and an LLM-judge track, combined 70/30 in both experiments. The judge is DeepSeek (deepseek-chat, temperature 0), which reviews each run’s self-reported summary, its narrative account and the code it wrote; one pass per run, averaged across runs. The judge comes from a provider not among the agents under test, so no agent is evaluated by a model from its own family.

Three decisions apply to both experiments. Validity is a gate rather than a scored dimension: a run that fails to deliver a well-formed prediction scores zero on the substantive dimensions, and that zero enters the mean, since excluding failures would reward an agent for failing. Full marks are calibrated to

an achievable reference rather than to perfection (§4.3.6). Weights are fixed before any score is computed and are not revised; the influence each dimension exerted is reported separately rather than tuned.

4.1.5 Difficulty recalibration and implementation

The first experiment was executed in full before the second was specified. Its outcome — every agent solving the task to numerical precision, so that the ranking reduced to price — is a measurement of the task rather than of the agents, and determined how the second was built. Experiment 2 retains the same four-stage architecture and the same experiment-agnostic dispatcher, and changes what those stages are given: the scientific task, the prompt variants, the scoring dimensions and their weights, the timeout, the working-directory handling, and the number of agents that could be measured (§4.3, §5.2). The new task introduces three difficulties: a measurement distribution whose obvious summary statistic is uninformative, an accuracy ceiling imposed by physics, and missing data that no prompt mentions.

The implementation separates data handling from execution. Each experiment is a class exposing a common interface and performing no modelling of its own, so the entire analytical burden falls on the agent. Agent dispatch is experiment-agnostic: one function receives a prompt and a working directory and invokes one interface headlessly. It was written for Experiment 1 and reused for Experiment 2 without modification.

4.2 Experiment 1

The training file is passed through in full; the test file has displacement, velocity and acceleration removed, leaving only timestamps. The generating parameters ($m = 1 \text{ kg}$, $c = 0.1 \text{ N s m}^{-1}$, $k = 1 \text{ N m}^{-1}$) are retained in memory for physics checks and never written to any file the agent can read.

The prompt withholds even the qualitative framing: the agent is told only that it observes the one-dimensional motion of a point mass and is asked not to assume a textbook system. Three deliverables are required: a self-contained script, a prediction file, and a JSON summary reporting the model class, governing equation, fitted parameters and reasoning.

Four objective dimensions are computed: result accuracy (per-column RMSE normalised by column standard deviation), scientific correctness (energy non-negative and non-increasing, amplitude bounded, equation of motion satisfied), run-to-run stability, and cost. Eight judged dimensions follow the rubric of Table 3.

Table 3. Scoring dimensions and weights, Experiment 1.

Track	Dimension	Weight	Definition of 100
Objective	Result accuracy	35%	Zero normalised RMSE against hidden targets
Objective	Scientific correctness	20%	All physics checks satisfied
Objective	Cost	10%	Cheapest agent
Objective	Run-to-run stability	5%	Zero spread across the three runs
AI judge	Data exploration	5%	Patterns examined before a model class chosen
AI judge	No shortcut or hardcoding	5%	Parameters derived by fitting, quality shown
AI judge	Validation rigour	4%	Genuine held-out validation
AI judge	Correct model identification	4%	Damped second-order system identified
AI judge	Self-consistency	4%	Summary, log and code agree
AI judge	Methodology soundness	4%	End-to-end workflow sound
AI judge	Reasoning clarity	2%	Precise and correctly-used terminology
AI judge	Code quality	2%	Readable and matching its description

4.3 Experiment 2

4.3.1 Parsing and label derivation

The source file is streamed rather than loaded. Species are derived from the mass field by nearest-neighbour matching in logarithmic space, necessary because the electron is three orders of magnitude lighter than the other species. Acceptance uses an absolute-or-relative tolerance: masses are stored rounded, so the electron appears as 0.0005 against a true 0.000511, and a 5% relative test alone would reject it and delete the entire class.

4.3.2 Withheld fields, splits and injected missingness

The mass determines the species and is the label itself. The predicted energy loss is computed from that mass and serves as ground truth for the estimation sub-task; releasing it would disclose the label. In a real analysis this column does not exist; one computes a predicted curve per species hypothesis instead, so withholding it also preserves the realism of the task.

A class-balanced sample is split per species: 5,000 training events at 1,000 per species and 1,500 test events at 300 per species. Real detector data are dominated by pions, but a balanced export keeps macro-F1 interpretable; the loss of realism is accepted deliberately.

Two forms of missingness are injected and never mentioned in the prompt. Individual measurements are deleted at a mean rate of 12%, with the per-event rate scaled by a uniform factor on (0, 2) so event quality is heterogeneous, subject to a floor of five surviving samples. Separately, 4% of events lose momentum entirely. Realised rates are 4.1% and 12.1% in training, 4.5% and 12.0% in test.

4.3.3 Dual-output task and prompt levels

The agent must produce two columns for every test event: `dedx_est`, a single best estimate of that event's true energy loss derived from its own repeated measurements, and `particle`, the predicted species. Both are scored, and the prompt says so.

The estimation column has no supervision anywhere in the training data. An agent cannot regress its way to it and must reason about what estimator the measurement process calls for; this is the mechanism by which producing a correct answer and understanding the measurement are separated. Bethe–Bloch, $\beta\gamma$, the Landau distribution, truncated means and the injected missing values are never mentioned.

The prompt exists in three variants, byte-identical outside a single Method block: level 1 gives no process guidance, level 2 names the four stages of a workable solution, level 3 adds a checklist of things worth verifying. The levels are a sampling condition rather than a controlled comparison, for reasons given in §6.5.

4.3.4 Selection of the reference estimator

The predicted energy-loss column is the ground truth against which `dedx_est` is scored. It is a generator quantity rather than a summary of the data: the provider has confirmed that it is the mean energy loss given by their Bethe–Bloch parametrisation at the particle's velocity, not a statistic of that event's fifty samples (§3.2). No discard fraction is therefore intrinsically correct, and the question becomes which sample statistic best recovers a noiseless target. Scanning the discard fraction from 0% to 80% answers it empirically: at 50% the mean ratio reaches 0.9925 and the relative RMSE its global minimum of 0.056, whereas at 35%, the figure quoted in the dataset specification, the estimator sits 8.5% high with an error of 0.108. We adopt the 50% truncated mean as an empirically selected reference estimator; it fixes the full-mark floor and nothing else, and the consequences of the choice are quantified in §6.4.2.

The comparison also establishes the character of the task. The arithmetic mean is not merely biased but uninformative: because the Landau tail falls as x^{-2} [14], the mean of fifty samples is dominated by its largest one or two entries and correlates with the true value at 0.003. The median reaches 0.861 and a truncated mean 0.94 or above.

4.3.5 Reference baseline

Task difficulty was calibrated before any paid run with a deliberately unremarkable solution: six hand-built features (log momentum, the truncated mean, the median, the arithmetic mean, the 25th percentile and the count of surviving samples) fed to a gradient-boosted tree at defaults. It achieves 0.593 accuracy and 0.592 macro-F1. It is never shown to an agent and takes no part in scoring; its purpose is to establish that the task sits in a usable range. Its confusion matrix confirms the task is well-posed, with errors concentrated between mass-adjacent species while physically implausible confusions are essentially absent.

4.3.6 Scoring and the calibration of full marks

Six objective and eight judged dimensions are combined 70/30 (Table 4). Full marks are calibrated to a measured floor rather than to perfection: the residual error of the reference estimator on the exported test set, 0.0596, defines 100 on the estimation axis. Because the target column carries no sampling noise of its own (§3.2), this residual is the irreducible noise of a fifty-sample statistic rather than an artefact of the target. This quantity, and only this quantity, depends on the choice of discard fraction; the per-run relative errors that enter the score are computed directly against the withheld prediction column and are unaffected. The decay constant is set so that the estimators agents actually reach for remain distinguishable rather than all rounding to zero. That the scale is achievable was confirmed in practice: one run reached 99.95.

Physics plausibility asks whether the errors are sensible rather than whether a conservation law holds. Among the events an agent got wrong, we measure how far apart the true and predicted species are in the mass ordering: adjacent confusion is the unavoidable consequence of converging Bethe–Bloch curves, whereas confusion spanning the full mass range indicates the relationship was never learned. Scoring over errors only keeps this dimension orthogonal to accuracy.

Table 4. Scoring dimensions and weights, Experiment 2.

Track	Dimension	Weight	Definition of 100
Objective	Classification quality	20%	Perfect macro-F1, chance-corrected
Objective	dE/dx estimation	20%	Measured noise floor (0.0596 relative error)
Objective	Cost	10%	Cheapest agent
Objective	Physics plausibility	8%	All errors between mass-adjacent species
Objective	Self-assessment calibration	7%	Claimed accuracy equals measured accuracy
Objective	Consistency	5%	Zero spread across the nine runs
AI judge	Estimator justification	6%	Estimator selected by measurement, not convention
AI judge	Data exploration	4%	Distribution examined before a summary chosen
AI judge	Data integrity check	4%	Missing values noticed without being told
AI judge	Validation rigour	4%	Genuine held-out validation
AI judge	Physics grounding	3%	Energy loss linked to momentum and mass
AI judge	Self-consistency	3%	Summary, log and code agree
AI judge	Methodology soundness	3%	End-to-end workflow sound
AI judge	Code quality	3%	Readable and matching its description

Compared with Experiment 1, the weight on result accuracy falls from 35% to 20% and is redistributed to the estimation axis and to self-assessment calibration, reflecting the change in what the task can discriminate.

5. Experimental Setup

A comparison of this kind must be fair, in that agents differ only in their own identity and not in the task, environment or credentials; credible, in that an observed difference exceeds the variation the same agent produces when nothing changes; and reproducible, in that a third party with the same files can obtain

the same task packages and audit every score. This section describes how each experiment met those requirements, and where it fell short.

5.1 Experiment 1

All five interfaces of Table 2 were run three times each on the identical prompt, giving fifteen runs. Each was invoked headlessly with interactive approval disabled, standard input closed, and a limit of 600 seconds; Claude Code additionally required its trust dialogue to be pre-accepted, since an unattended run would otherwise block on it. Every subprocess received a restricted environment containing base system variables plus that agent’s own credentials and nothing else, so that one agent could not read another provider’s key. Each run executed in its own subdirectory with a fresh copy of the task files.

Cost is recorded per run in CNY. Because the interfaces expose usage incompatibly, four extraction paths were required: Claude Code reports a total cost directly; Qwen Code’s token counts are read from its output and priced at published rates; Codex CLI’s usage events distinguish cached from uncached input, priced separately; Kimi CLI, which exposes no usage, is measured from the account balance before and after each run; CodeBuddy CLI is priced from the credits it reports.

For every run we retain the script, the predictions, the self-reported summary, and a log with return code, elapsed time, complete output streams and extracted usage. All scores are recomputed from these artefacts rather than recorded at run time, so any score can be re-derived or challenged.

5.2 Experiment 2

Four interfaces completed the experiment; Kimi CLI could not be run and is excluded from all quantitative results (§8). The protocol of §5.1 applies with three additions. Each working directory is deleted and recreated before task files are written rather than merely overwritten, since otherwise a repeated run would land beside the previous attempt’s script and predictions, a form of contamination that produces no error and no warning, and a particular hazard because three prompt levels share one directory layout. The prompt each run received is stored in its log rather than written into the working directory, an earlier arrangement having led one interface to read it as input. The wall-clock limit was raised, since fitting and validating a classifier over five thousand events requires materially more turns.

5.2.1 Sampling design

Each agent was run nine times: three prompt levels by three repetitions, giving thirty-six runs. Nine rather than three follows from a measurement: a 3×3 pilot placed the run-to-run standard deviation at 0.070 in accuracy, so averaging three runs leaves a standard error of 0.041 and separates agents only if they differ by roughly 0.11, wider than the entire spread observed in Experiment 1. Nine runs halve the resolvable difference to approximately 0.065 and give every agent its own variance estimate.

5.2.2 Cost accounting

Agentic runs re-send their accumulated context on every turn, so prompt-cache hit rates above 90% are routine. Pricing every input token at the standard rate overstated one agent’s cost by a factor of three while another interface’s extractor had always separated cached from uncached input, so the two were being compared on different terms and only one was penalised. All extractors now apply the same three-way split of uncached input, cached input and output.

5.2.3 Reproducibility and known threats

The exported task packages are deterministic: a single seed governs the class-balanced sampling, the split, the deletion of measurements and the removal of momentum values, so the identical files can be regenerated from the source data. Three threats remain. Three of the five interfaces ran on their default

model, which changes with interface version; we record the models invoked but did not fix them. Two failed runs were re-executed and their originals overwritten, so reported validity rates are marginally optimistic. And identical code produced opposite outcomes for one interface depending on whether the notebook kernel had been launched from a shell or spawned by an IDE, because the former inherits a controlling terminal; we resolved this with an explicit pseudo-terminal, but the episode indicates that execution details belong in a reported method.

5.2.4 Protocol deviations

Because a re-executed run replaces a failure with a success, every departure is recorded in Table 5 rather than described in prose. Two runs were re-executed after failing, in both cases under an identical prompt and protocol, and in both cases the originals were overwritten rather than archived. This is the one departure we would not repeat: the failure rates reported in §6 are consequently marginally optimistic and the raw rates cannot now be recovered. Neither changes the ordering of §6.2, since both agents lead the third-placed agent by more than the value of a single run.

One change of procedure was made during pilot work and is recorded because it could otherwise be mistaken for a deviation within the reported runs. The prompt was initially written into each working directory as a text file; one interface read it as though it were input data, and the prompt was thereafter stored in the run log instead. Inspection of the released artefacts confirms that all thirty-six scored runs used the later procedure: none contains a prompt file, every log carries the prompt it received, and no output stream refers to such a file. Experiment 1 was executed without deviation.

Table 5. Protocol deviations, Experiment 2.

Run ID	Agent	Observed problem	Re-run	What changed	Original kept
L2_r2	Claude Code	Ended its turn awaiting a background-task notification that cannot arrive in a headless session; return code 0, script written, no predictions	Yes	Nothing; identical prompt and protocol	No
L3_r2	CodeBuddy CLI	Failed to produce a complete set of deliverables; artefacts overwritten on re-execution before the failure mode was recorded	Yes	Nothing; identical prompt and protocol	No
all nine	Kimi CLI	Silent hang at start-up: no output, no processor activity, no network connection, until the time limit	Repeatedly attempted	Non-interactive flags, pseudo-terminal, execution from a real terminal, alternative models, replacement key, upgrade	n/a — excluded (§8)

6. Results

Results are ordered by the questions a reader will ask in sequence rather than by the order in which the experiments were run.

6.1 The control task does not discriminate

All five agents produced numerically accurate predictions and passed the objective physics checks. Result accuracy, scientific correctness and stability are identical to numerical precision across the field, at 99.999925, 100 and 100 respectively, so sixty of the seventy objective percentage points produced no differentiation whatsoever. What remained was cost, which spans an order of magnitude, together with deductions on three judged dimensions. The resulting ranking is a cost ranking with minor adjustments, and it is fragile: correcting a single cache-accounting convention moved one agent from fourth place to third.

Five judged dimensions were identical across all five agents. Absence of shortcuts scored 10 for every agent, correct model identification and self-consistency scored 1 for every agent, and validation rigour scored 6 for every agent, a uniform 6 recording that no agent performed genuine held-out validation.

Table 6. Experiment 1 final scores and the dimensions on which agents differ.

Agent	Final	Cost	Data expl.	Methodology	Code quality
Kimi CLI	97.56	92.95	1.00	9.67	10.00
CodeBuddy CLI	96.73	100.00	0.67	10.00	10.00
Qwen Code	96.36	88.98	1.00	8.67	9.00
Codex CLI	95.63	77.59	1.00	9.00	9.67
Claude Code	89.79	13.88	1.00	10.00	10.00

Final and Cost are on a 0–100 scale; Data expl. is a 0–1 binary criterion and Methodology and Code quality are on the judge’s native 0–10 scale. All judged values are rescaled to 0–100 before the weighted sum. Accuracy, scientific correctness and stability are omitted because all five agents are identical on them.

6.2 The redesigned task discriminates, along an axis accuracy cannot see

In the second experiment the two axes separate cleanly. Estimation scores span 19.8 to 86.6, a factor of more than four, while classification scores span only 48.4 to 53.6. Kimi CLI is absent for the reasons given in §8. The scores reported here are conditional on the reference estimator adopted in §4.3.4; §6.4.2 establishes the range over which they are invariant to that choice.

The clearest single piece of evidence is Qwen Code. Its classification accuracy of 0.60 is level with our own reference baseline of 0.593, yet its estimation score is 19.8, the lowest of the field. Qwen Code achieved baseline-level classification performance but failed to recover a low-bias estimator of the withheld dE/dx quantity; this failure would not have been visible from classification accuracy alone. Its errors are moreover systematic rather than random: across nine runs it cycled between a median, an interquartile-range trimmed mean and a light truncation, and every run overestimated, with ratios between 1.20 and 1.38. Repeating the same biased procedure would not remove this systematic error.

At the other end, Claude Code identified the correct family of estimator in all nine runs, naming the Landau distribution, delta-ray production and the truncated mean as the detector-physics standard. Eight of the nine applied the conventional practice of retaining the lowest 60% of samples, scoring between 83 and 87; the one run that retained the lowest 50% scored 99.95, landing essentially on the reference floor. Its residual shortfall is not a failure of physics but of empiricism: it applied a textbook fraction rather than measuring which fraction this data called for. Claude Code and Codex CLI are not separated at this sample size (§6.4.1).

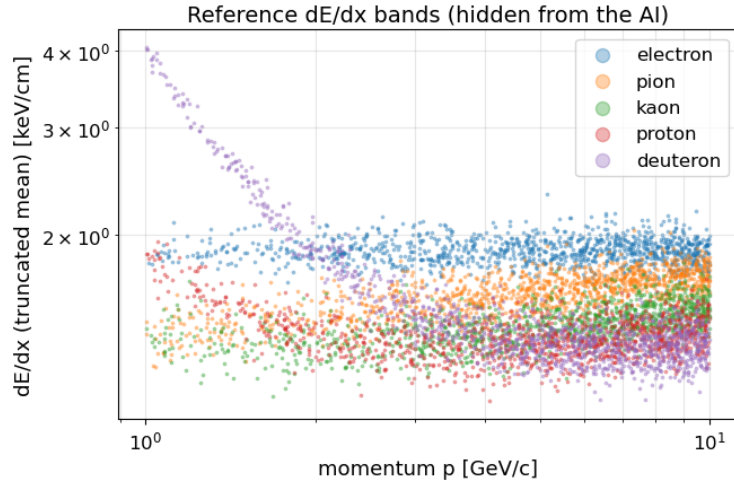


Fig. 2 Reference dE/dx against momentum p for the five species, used for display only and never shown to an agent. The pion, kaon and proton bands converge as p rises, which limits achievable classification accuracy.

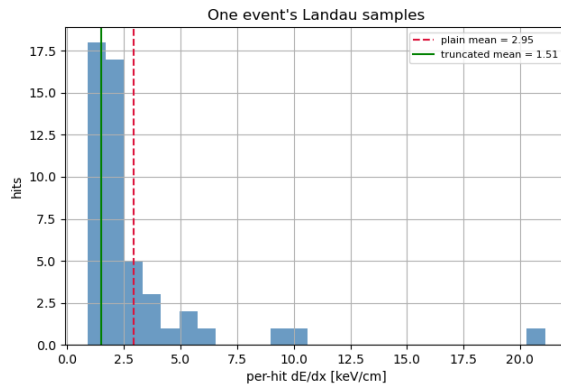


Fig. 3 The fifty samples of a single event. A few large deposits beyond 20 keV/cm place the arithmetic mean at 2.95 against a truncated mean of 1.51.

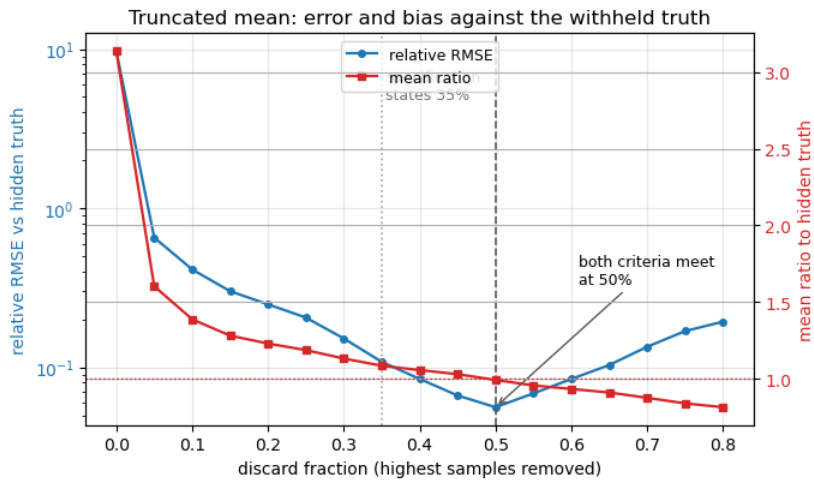


Fig. 4 Relative error of the truncated mean against the withheld prediction column, as a function of the discard fraction.

Table 7. Experiment 2 final scores and per-dimension results. Objective dimensions are shown on a 0–100 scale; judged dimensions are shown on their native 0–1 or 0–10 scales.

Dimension	CodeBuddy	Claude	Codex	Qwen	Weight
Final score	77.81	71.21	69.98	52.77	—
Classification quality	53.64	53.49	52.95	48.37	20%
dE/dx estimation	68.47	86.63	68.54	19.81	20%
Cost	100.00	7.97	36.15	61.92	10%
Physics plausibility	83.70	83.21	82.38	82.21	8%
Self-assessment	74.23	60.50	82.60	82.67	7%
Consistency	97.96	98.79	99.05	96.30	5%
Estimator justification	7.78	7.44	6.89	4.11	6%
Data exploration	0.89	1.00	1.00	0.44	4%
Data integrity check	1.00	1.00	1.00	0.44	4%
Validation rigour	8.11	7.56	8.11	6.78	4%
Physics grounding	1.00	1.00	0.56	0.11	3%
Self-consistency	1.00	1.00	1.00	1.00	3%
Methodology soundness	8.44	8.22	7.56	5.11	3%
Code quality	8.67	8.67	8.11	7.22	3%

The 0–1 judged dimensions are binary criteria; all judged values are rescaled to 0–100 before the weighted sum.

6.3 What decides the ranking

A weighted total does not reveal what produced it. A dimension influences the ranking in proportion to its weight multiplied by the spread observed on it, and by that measure two dominate: estimation moves the final score by 13.4 points and cost by 9.2, together 62% of all differentiation on 30% of the nominal weight. Classification, despite carrying 20% of the weight, contributes 1.05 points, the four agents differing by only 5.3 points on it. This is an honest null result about the task rather than a defect of the metric. Three further dimensions produced almost no differentiation (physics plausibility at 0.12, consistency at 0.14 and self-consistency at 0.00), so 16% of the weight did no work.

The margin at the top deserves stating plainly. CodeBuddy CLI leads or ties on classification, methodology and estimator justification as well as being cheapest, so its lead is broad-based; but cost alone contributes 9.2 points of separation against Claude Code’s 3.6-point advantage on estimation, and removing the cost dimension would reverse the top two positions.

Table 8. Dimension influence: weight multiplied by observed spread, in final-score points.

Dimension	Minimum	Maximum	Weight	Points swung
dE/dx estimation	19.81	86.63	20%	13.36
Cost	7.97	100.00	10%	9.20
Physics grounding	11.11	100.00	3%	2.67
Data exploration	44.44	100.00	4%	2.22
Data integrity check	44.44	100.00	4%	2.22
Estimator justification	41.11	77.78	6%	2.20
Self-assessment	60.50	82.67	7%	1.55
Classification quality	48.37	53.64	20%	1.05
Methodology soundness	51.11	84.44	3%	1.00
Validation rigour	67.78	81.11	4%	0.53
Code quality	72.22	86.67	3%	0.43
Consistency	96.30	99.05	5%	0.14
Physics plausibility	82.21	83.70	8%	0.12

Dimension	Minimum	Maximum	Weight	Points swung
Self-consistency	100.00	100.00	3%	0.00

6.4 Robustness of the ranking

6.4.1 Sampling uncertainty and rank stability

Several differences above are smaller than the resolution the sampling design supports, so point estimates alone are not a sufficient basis for a ranking. We resampled each agent’s nine runs with replacement 10,000 times and recomputed the entire pipeline inside every replicate. All four agents are resampled within the same replicate, so the cost score, which is relative across agents, and the consistency score, which is a function of the resampled spread, are recomputed each time rather than held fixed. The nine runs are three at each of the three prompt levels, and the resampling is not stratified by level: a replicate may over- or under-represent a level, so the intervals absorb between-level variation as well as run-to-run noise and are conservative in that respect. Because each level contributes only three runs, we report these as 95% run-resampling intervals over the runs actually executed, not as confidence intervals for performance over a broader population of tasks or prompts. CodeBuddy CLI finishes first in every replicate and its interval is disjoint from the second-placed agent’s, so the lead is not an artefact of which runs were sampled. Second and third place are not resolved: Claude Code leads Codex CLI by 1.23 points, their intervals overlap over most of their length, and Claude Code finishes ahead in 81.3% of replicates. We report these two as indistinguishable rather than as ranked. Qwen Code is last in every replicate.

At dimension level the intervals confirm quantitatively what §6.3 inferred. The classification intervals of the top three overlap almost entirely, so that dimension does not separate them at all. The estimation axis behaves oppositely: Claude Code’s interval is disjoint from Qwen Code’s and overlaps CodeBuddy CLI’s only at its lower edge. The widest interval anywhere is CodeBuddy CLI’s estimation score, reflecting genuine variation in which estimator it selected from run to run.

Table 9. Bootstrap 95% run-resampling intervals over 10,000 resamples of the nine runs per agent.

Agent	Final score	Classification	dE/dx estimation	Cost
CodeBuddy CLI	77.81 [74.34, 80.92]	53.6 [52.4, 55.0]	68.5 [50.6, 84.4]	100.0 [100, 100]
Claude Code	71.21 [69.65, 72.71]	53.5 [52.7, 54.3]	86.6 [84.0, 90.4]	8.0 [6.5, 9.7]
Codex CLI	69.98 [67.81, 72.31]	52.9 [52.3, 53.5]	68.5 [59.4, 78.9]	36.2 [29.4, 44.2]
Qwen Code	52.77 [50.04, 55.79]	48.4 [45.9, 50.7]	19.8 [13.5, 25.3]	61.9 [49.8, 78.0]

Table 10. Rank stability across the same 10,000 replicates.

Quantity	Probability
CodeBuddy CLI finishes first	1.000
Observed ordering preserved	0.813
Claude Code ranks above Codex CLI	0.813
Codex CLI ranks above Claude Code	0.187
Every agent ranks above Qwen Code	1.000

Only two orderings occur: the observed one, in 81.3% of replicates, and the same ordering with second and third exchanged. First and last place never change.

6.4.2 Sensitivity to the reference estimator

The discard fraction defining the reference estimator was selected empirically and is therefore a free parameter of the scoring scheme. It enters in one place only: the full-mark floor. We recomputed the floor at discard fractions from 35% to 60%, rescored every run’s estimation dimension against each, and

recomputed the ranking (Table 11). The floor traces a smooth minimum at 50% and the curve is close to symmetric about it (0.0673 at 45% against 0.0676 at 55%), the shape of a bias–variance trade-off rather than of a threshold, as expected when the target is a noiseless generator quantity rather than a statistic of the samples (§3.2).

The ordering of agents on the estimation axis is invariant across the entire range. Absolute scores move substantially — Claude Code ranges from 86.6 at the adopted setting to 100.0 at 35%, where its own estimator outperforms the reference and its score is censored at the ceiling — so the analysis is not vacuous; it is the ordering that is stable. The final ranking is unchanged over 40–60%. Only at 35% do Claude Code and Codex CLI exchange second and third, by 1.07 points, which is smaller than the resolution the sampling design supports. The results of §6 should be read as conditional on the reference estimator lying at or above a 40% discard; within that condition they do not depend on its exact value.

Table 11. Sensitivity of the final scores and ranking to the discard fraction. The adopted setting is 50%.

Discard	Floor	Claude	CodeBuddy	Codex	Qwen	Final ranking
35%	0.1185	73.88	81.97	74.95	58.53	CodeBuddy CLI > Codex CLI > Claude Code > Qwen Code
40%	0.0881	73.72	80.93	73.19	55.89	CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code
45%	0.0673	72.09	78.84	71.03	53.67	CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code
50%	0.0596	71.21	77.81	69.99	52.77	CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code
55%	0.0676	72.12	78.88	71.06	53.70	CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code
60%	0.0859	73.62	80.77	73.00	55.67	CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code

6.5 Prompt specificity

Across four agents and three levels, prompt detail produced no resolvable effect on classification accuracy, cost, or detection of the missing values: for each, the spread between level means is smaller than the run-to-run variation within a single level. A monotonic cost trend visible in an unreplicated first pass vanished once replicated, which is the clearest available demonstration of why replication was necessary.

On the estimation axis the pattern differs, though it remains exploratory. All four agents scored lower at the most detailed level than at the vaguest, a paired within-agent difference averaging 15.7 points. The effect-to-noise ratio of 1.29 classes this as unresolvable; the paired comparison is less conservative than the pooled statistic, since the latter allows one agent’s instability to raise the noise floor for the other three. A sign test on four of four agents moving in the same direction gives $p = 0.125$. At this number of agents that does not support a stable prompt effect, and we report the pattern as exploratory rather than as a finding.

Detection of the missing values barely moved even though the most detailed prompt raises them explicitly, because of a ceiling effect: three agents score 1.00 at every level and one scores 0.44 at every level.

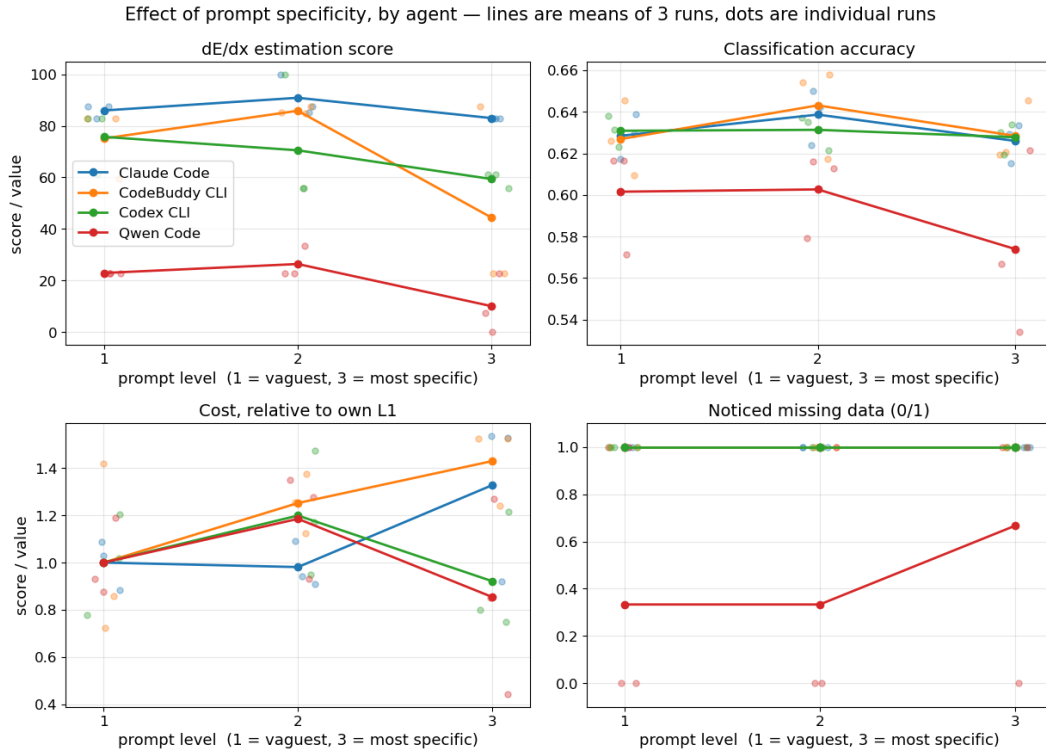


Fig. 5 Effect of prompt specificity by agent. Each agent contributes three runs per level ($n = 3$ per cell, 36 runs total): lines join the means, light points are individual runs. The four panels use different quantities and scales and are not comparable with one another.

Table 12. Prompt-level comparison. An effect-to-noise ratio below approximately 2 indicates the levels cannot be told apart at this sample size.

Metric	L1	L2	L3	Effect / noise	Agents improving
Classification accuracy	0.622	0.629	0.614	0.76	1 / 4
Cost, relative to own L1	1.00	1.15	1.13	0.62	2 / 4
Noticed missing data (0/1)	0.833	0.833	0.917	0.29	1 / 4
dE/dx estimation score	64.9	68.4	49.2	1.29	0 / 4

7. Discussion

7.1 Benchmark design implications

The contrast between the two experiments is consistent with a single reading: a benchmark measures only what its task can make agents fail at. The oscillator has one correct answer that any competent agent finds, so a scoring scheme built to assess scientific capability ended up assessing token pricing, and one accounting convention was enough to reorder the field. The second task was built with three deliberate difficulties — a heavy-tailed measurement distribution, a physically imposed accuracy ceiling, and undisclosed missing data — together with a change in the output specification, and the fourfold spread on the estimation axis was associated with that redesign. We did not run an ablation: because all four changes were introduced simultaneously, the individual contribution of each cannot be identified from these data. What the comparison supports is that the redesign as a whole restored discrimination, not which component was responsible.

The two abilities the second task separates are not the same ability at different strengths. Every agent classified at roughly the same level, close to the performance plateau observed for the tested baseline

and agents — the bootstrap intervals of the top three overlap almost entirely (§6.4.1) — while they differ by more than fourfold on whether they worked out that a Landau distribution demands an aggressive truncation. Accuracy alone would have shown none of this, and this second ability is likely to be relevant to analyses where no labelled target is available — the capability the benchmark is intended to approximate, though we ran no cross-task or cross-dataset test of transfer.

Domain knowledge helps but is not strictly required. Codex CLI articulated the relationship between energy loss and mass in only five of nine runs, yet matched CodeBuddy CLI’s estimation score; a purely statistical route reaches a competent estimator. It does not reach Claude Code’s, and Claude Code’s own shortfall came from applying a domain convention rather than measuring which parameter the data preferred. The distinction between invoking domain knowledge and testing it against the data is what the estimator-justification dimension attempts to capture.

7.2 Validity of LLM-based judging

That dimension also exposed a weakness in LLM-based judging, and in addressing it we changed a scoring rule after results had been seen. Our first pass awarded 10 out of 10 to a run that asserted the conventional 60% truncation and 9 out of 10 to the run that measured its way to the better value, because the former was written in fluent and correct domain language; the rubric had asked for evidence of measurement but did not enforce it. We tightened the criterion by requiring the comparison itself to be visible and by stating that correct terminology is not evidence of measurement. All thirty-six runs were then scored again under both wordings. The revision lowers every agent’s estimator-justification score by between 0.33 and 0.78 points on a ten-point scale and moves final scores by less than 1.5 points; neither the ordering on that dimension nor the final ranking changes, both rubrics yielding CodeBuddy CLI > Claude Code > Codex CLI > Qwen Code. Two caveats apply: the two passes are separate calls to the judge, so the differences include its own variability across all eight dimensions; and both use the same judge model, so this tests the rubric, not the judge. The general caution stands: a well-written justification for a mediocre decision is the hardest thing for a judge to detect, and judged dimensions should be treated as softer evidence than deterministic ones.

7.3 Capability–cost trade-off

Capability and cost are better read as two axes than as one number. Figure 6 places each agent by its estimation score and mean cost per run. Qwen Code is dominated: CodeBuddy CLI is both cheaper and more accurate. Codex CLI lies on the frontier by 0.07 points of estimation score, well inside its bootstrap interval, so the practical choice is between CodeBuddy CLI and Claude Code at roughly twelve times the cost for an estimation score higher by 18 points. The 10% weight given to cost in §6 is a policy choice rather than a property of scientific capability, and happens to fall close to the break-even point between those two options; a reader who would weight cost differently should read the frontier rather than the ranking, and §6.4.1 and §6.4.2 bound how far the ranking moves under changes to the scoring.

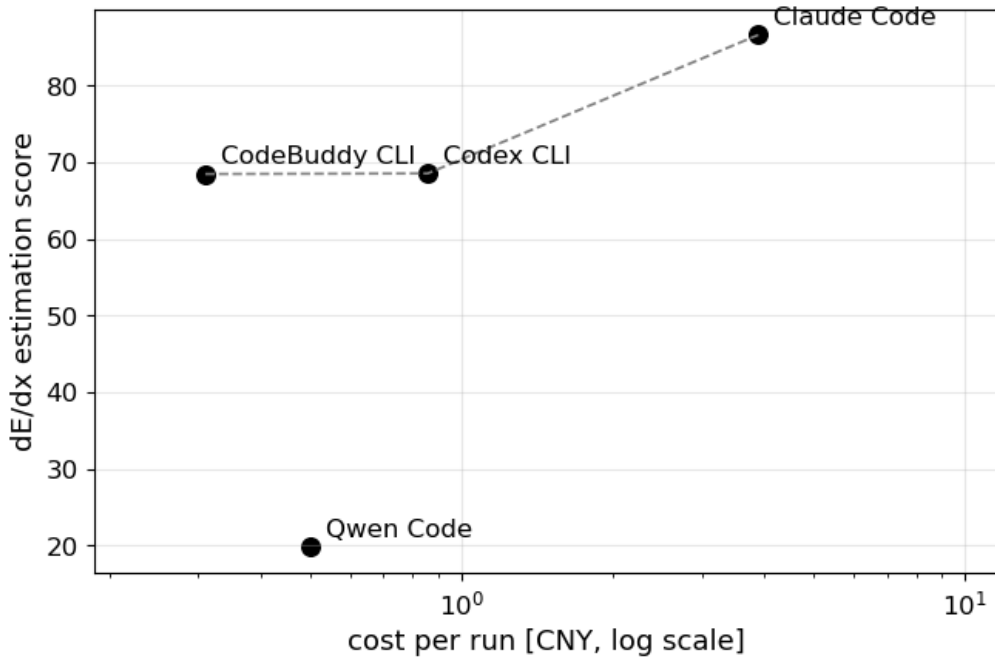


Fig. 6 Estimation score against mean cost per run, on a logarithmic cost axis. The dashed line marks the Pareto frontier.

8. Limitations

The findings apply to supervised physics tasks with hidden ground truth at the scale of a few thousand samples, executed by general-purpose coding agents through a command-line interface, using model versions and prices current in July 2026. Three execution-level threats — unpinned model defaults, two re-executed runs, and sensitivity to the launch environment — are documented in §5.2.3 and §5.2.4. The boundaries below concern the conclusions.

These results do not constitute a general capability ranking or a procurement recommendation. The two experiments produce different leaders, which is itself evidence of how strongly a ranking depends on the task it is derived from. What the study supports is a claim about how benchmarks should be constructed, not about which tool to use.

Statistical power is limited. The measured run-to-run standard deviation of 0.070 in accuracy means that even nine runs resolve only differences above approximately 0.065, and several gaps reported in §6.2 and §6.3 are smaller than that. Bootstrap resampling places the observed ordering at 81.3% probability: first and last place are stable across every replicate, but second and third exchange in the remainder, so Claude Code and Codex CLI are not separated at this sample size (§6.4.1). The prompt-specificity observation of §6.5 rests on four agents and does not attain conventional significance.

The reference statistic is a choice of ours rather than a property of the data. The provider has confirmed that the withheld column is the mean predicted by their Bethe–Bloch parametrisation and not a statistic of the fifty samples (§3.2), so no discard fraction is intrinsically correct; we adopted 50% because two independent criteria converge there, against the 35% quoted in the dataset specification (§4.3.4). The choice enters the scoring in one place only, the full-mark floor. Because it would otherwise be the largest single source of doubt about the Experiment 2 ranking, its consequences are quantified in the Results rather than deferred here: §6.4.2 shows the ordering on the estimation axis invariant from 35% to 60% and the final ranking invariant over 40–60%.

The judged track is weaker evidence than the objective track. It relies on a single model applying a rubric we wrote, and that rubric was revised once during analysis after it demonstrably failed to enforce its own stated criterion (§7). A reader who discounts judged dimensions entirely would still be left with the study’s principal conclusions intact.

One agent could not be measured at all. Kimi CLI hangs silently in headless mode — no output, no processor activity, no network connection — until the wall-clock limit expires. We eliminated data size, prompt length, working directory, all four models the interface offers, its configuration, credentials, terminal type and interface version as causes; the fault reproduces in an empty directory with a five-word prompt and is intermittent rather than deterministic. In the runs that did complete, its methodology was the strongest we observed: it alone compared candidate estimators empirically rather than adopting a convention, identified the band structure correctly, and detected the injected missing values without being told they were there. Model capability and engineering maturity are evidently independent properties, and for unattended scientific work the latter is a precondition rather than a convenience: a silent hang is the most damaging failure mode available, because it consumes the entire time budget while leaving no diagnostic trace.

9. Conclusion

Constructing an effective physics benchmark for AI agents depends less on the sophistication of the scoring scheme than on whether the task contains a way to fail that the headline metric cannot reveal. Our first experiment had no such failure mode and consequently measured price: five agents were identical to numerical precision on accuracy, scientific correctness and stability, and the ranking turned on token accounting. Our second was built with one, and separated the four interfaces that completed Experiment 2 by more than a factor of four on the estimation dimension.

Three design principles follow. A task should contain a failure path invisible to its principal metric — here, an estimation sub-task with no supervision anywhere in the training data, on which one agent exceeded our own physics baseline at classification while scoring near the bottom. Full marks should be calibrated to a measured, achievable floor rather than to theoretical perfection, since irreducible measurement noise otherwise compresses every agent into the same narrow band. And a ranking should be published alongside an analysis of which dimensions produced it, because a weighted total cannot distinguish a lead earned on scientific capability from one earned on price. The second of these carries a caveat: the target the floor approximates is a noiseless generator quantity, so no estimator of it is prescribed by the data, and the one we adopted was selected empirically; §6.4.2 bounds the consequence.

The most honest summary of our results is a set of measurements rather than a verdict. The agent with the highest dE/dx estimation score, 86.6 against 68.5 for the next, and the only one to identify the correct family of estimator in all nine of its runs, did not rank highest overall and cost roughly twelve times as much as the agent that did. On the classification task those two are indistinguishable, their bootstrap intervals overlapping almost entirely. None of this constitutes a recommendation between them. It indicates that producing a correct answer and reconstructing the quantity being measured are, at present, separable abilities, and that a benchmark will only report the second if its task is designed to require it.

Data and Code Availability

The oscillator dataset of Experiment 1 is public and is cited as [12]. The dE/dx sample of Experiment 2 was supplied by our mentor’s group and is not ours to redistribute; it is identified by the file name `dedx_sample_ecal_flatpT_pikpde_28_100000.txt`, a size of 27,248,608 bytes and the SHA-256 digest 30509f9e14e49af650b358951d9dea58f71bcffb387d68282ae2e3683121664a. Requests for the sample should be directed to the provider. Appendix B gives a specification sufficient to reproduce the experiment on an equivalent sample.

The task-construction, scoring, AI-judge and bootstrap code is contained in a single notebook, `benchmark.ipynb`, released together with the prompt templates and task specifications at <https://github.com/jby235278-wq/PhysArena> (branch `main`). The repository also provides a runner interface that reproduces the execution protocol of §4.1. The per-run logs and the artefacts each agent submitted are not committed and are available from the authors on request; the evaluation file carrying the withheld particle, mass, `dedx_pred` and label columns is deliberately not published, so that the task remains blind for anyone who wishes to be measured against it. The random seeds are fixed in the notebook, so the exported task packages regenerate byte-for-byte from the source file; the agent runs themselves are not deterministic and are supplied as logs rather than as a re-runnable script.

Acknowledgments

We thank Gunther Roland for supervision and for providing the dE/dx sample, and Alex Cai for guidance throughout the project.

References

- [1] Chen et al., “ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery,” ICLR 2025, arXiv:2410.05080.
- [2] Qiu et al., “PRBench: End-to-end Paper Reproduction in Physics Research,” arXiv preprint arXiv:2603.27646, 2026.
- [3] Faroughy et al., “Collider-Bench: Benchmarking AI Agents with Particle Physics Analysis Reproduction,” arXiv:2605.13950, 2026.
- [4] Chung et al., “Theoretical Physics Benchmark (TPBench) — a Dataset and Study of AI Reasoning Capabilities in Theoretical Physics,” arXiv:2502.15815, 2025.
- [5] Xu et al., “PhySense: Principle-Based Physics Reasoning Benchmarking for Large Language Models,” arXiv:2505.24823, 2025.
- [6] Wang et al., “PhysUniBench: A Multi-Modal Physics Reasoning Benchmark at Undergraduate Level,” arXiv:2506.17667, 2025.
- [7] Zheng et al., “NewtonBench: Benchmarking Generalizable Scientific Law Discovery in LLM Agents,” ICLR 2026, arXiv:2510.07172.
- [8] Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” ICLR 2024, arXiv:2310.06770.
- [9] White et al., “LiveBench: A Challenging, Contamination-Limited LLM Benchmark,” ICLR 2025 Spotlight, arXiv:2406.19314.
- [10] Liang et al., “Holistic Evaluation of Language Models (HELM),” Transactions on Machine Learning Research, 2023, arXiv:2211.09110.
- [11] Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” NeurIPS 2023, arXiv:2306.05685.
- [12] cici118, “Damped Harmonic Oscillator Dataset,” Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/cici118/damped-harmonic-oscillator>. Accessed: 20 August 2026.
- [13] H. Bethe, “Zur Theorie des Durchgangs schneller Korpuskularstrahlen durch Materie,” *Annalen der Physik*, vol. 397, no. 3, pp. 325–400, 1930, doi:10.1002/andp.19303970303.
- [14] L. D. Landau, “On the energy loss of fast particles by ionization,” *Journal of Physics (USSR)*, vol. 8, no. 4, pp. 201–205, 1944.
- [15] Provider pricing and billing documentation for the five interfaces; individual sources, rates and the conversion formula are listed in Appendix C.

Appendix A. First event of the dE/dx sample

The file stores one five-field header row per event followed by that event's fifty measurements, one per row. The header of the first event is reproduced verbatim; its fifty values are then given in a compact grid, read in file order.

```
50;3.08491;1.31761;0.937;0
```

2.56	2.22	3.06	1.16	1.57	1.37	1.48	7.17	4.66	1.74
1.21	1.17	2.14	2.10	1.02	1.12	1.79	1.45	1.08	1.03
4.52	1.67	1.53	1.17	0.928	1.81	1.75	1.54	1.65	2.26
1.78	1.52	1.62	1.56	2.05	1.47	1.09	3.35	2.09	0.896
1.91	2.56	0.918	1.11	1.52	1.99	1.60	2.44	2.39	1.19

Header fields, in order: $n_{\text{hits}} = 50$; momentum = 3.08491 GeV/c; dE/dx prediction = 1.31761 keV/cm (withheld); mass = 0.937 GeV/c², identifying the particle as a proton (withheld); label = 0 (withheld). For this event the arithmetic mean of the fifty measurements is 1.900 and the median 1.610, against a true value of 1.318; the mean of the lowest twenty-five is 1.268. The single largest measurement, 7.17, is more than five times the true value and alone shifts the arithmetic mean by roughly 0.12.

Appendix B. Reconstruction procedure

The generator was not made available to us. The procedure below states what the released file and the provider's description establish, and marks what remains undisclosed. A reader wishing to reproduce the experiment on an equivalent sample can implement this specification; reproducing our numbers exactly requires the original file, identified by name, size and checksum in the Data and Code Availability statement.

```
for each of N = 100,000 events:
  species <- uniform over {e, pi, K, p, d}
  m       <- mass(species)           # stored rounded to 3-4 decimals
  pT      <- flat over [1, 10] GeV/c # eta = 0, so p = pT
  bg      <- p / m
  mu      <- f(bg)                   # Bethe-Bloch parametrization; constants not
disclosed
for i in 1..50:
  x_i <- Landau sample about mu      # width not disclosed
  emit header: 50 ; p ; mu ; m ; label
  emit x_1 ... x_50, one value per row
```

Confirmed by the provider: the momentum field stores the transverse momentum with pseudorapidity set to zero, so $p = p_T$; and the predicted energy-loss column is the mean given by their Bethe–Bloch parametrisation at the particle's velocity, not a statistic of the fifty samples. Verified empirically from the released file: the mean of the lowest 50% of the samples reproduces the predicted value with mean ratio 0.9925 and relative RMSE 0.056, and no other discard fraction satisfies both criteria simultaneously (§4.3.4); the arithmetic mean correlates with it at 0.003, consistent with a tail falling as x^{-2} ; species proportions are equal to within 1.2%; the momentum field is uniform over 1–10 GeV/c; and every event carries exactly fifty measurements.

Not recoverable from the released file: the Bethe–Bloch parameterisation [13] and its detector constants, the width parameter of the Landau sampling, the random seed, and the meaning of the numeric tag 28 in the filename. The binary header field named label is not used in either task because its semantics are unknown; the five-species target is derived solely from the withheld mass field.

Appendix C. Cost accounting

Each interface reports usage in a different form, so a single conversion was applied to bring all figures to CNY per run. Where an interface reports a price directly, that price is used and no rate table is applied; where it reports only token counts, the published rates below are applied.

For the two interfaces priced from token counts, the cost of a run is

$$\text{cost} = (\text{uncached input} / 10^6) \cdot r_{\text{in}} + (\text{cached input} / 10^6) \cdot r_{\text{cached}} + (\text{output} / 10^6) \cdot r_{\text{out}},$$

with uncached input taken as $\max(0, \text{input tokens} - \text{cached tokens})$. Rates quoted in USD are multiplied by a USD→CNY rate fetched at scoring time, with 7.2 as a fallback if the rate service is unreachable. Cache-discounted input is priced separately rather than at the full input rate; an earlier pass that did not do so overstated one agent’s cost by roughly a factor of three and changed its position in the Experiment 1 ranking (§6.1).

Table 13. Basis of the cost figure for each interface.

Interface	Basis of the cost figure	Rates applied	Source
Qwen Code	Token counts from the run log, priced from published rates.	CNY per 10 ⁶ tokens: input 4.00, cached input 0.80, output 16.00.	Alibaba Cloud Model Studio model pricing, accessed August 2026.
Codex CLI	Token counts from the run log, priced from published rates, then converted to CNY.	USD per 10 ⁶ tokens for gpt-5.3-codex, API-key auth, standard tier: input 1.75, cached input 0.175, output 14.00.	OpenAI pricing page, accessed August 2026.
CodeBuddy CLI	Credits consumed, read from the run log.	99 CNY per 2,000 credits on the Personal Pro plan, i.e. 0.0495 CNY per credit.	CodeBuddy account and pricing documentation, accessed August 2026.
Claude Code	The interface reports total_cost_usd for each run in its JSON output; that figure is used directly and converted to CNY.	None applied; the provider performs the pricing.	Interface output.
Kimi CLI	No usage is exposed in headless mode. Cost is the decrease in the Moonshot account balance across the run, already in CNY.	None applied; the provider performs the pricing.	Moonshot account balance API.

Rates are those in force at the time of the runs. Because two interfaces are priced by the provider and three from published rate tables, costs are comparable in currency but not in billing model; the cost dimension should be read accordingly.